Sprint 1 — Backend base + módulo Employees

Fecha: 17/04/2026

Objetivo

Construir un backend funcional con FastAPI conectado a PostgreSQL e implementar el primer módulo real del sistema: gestión de empleados con operaciones CRUD completas y validaciones básicas.

Resultado

Backend operativo con API documentada en Swagger, persistencia en base de datos y módulo de empleados funcional y coherente con un entorno tipo ERP.

Estructura del proyecto

```
backend/
├── app/
│   ├── main.py
│   ├── config.py
│   ├── db.py
│   ├── models/
│   │   ├── user.py
│   │   └── employee.py
│   ├── schemas/
│   │   └── employee.py
│   └── crud/
│   └── employee.py
├── venv/
├── requirements.txt
├── .env
```

Tecnologías utilizadas

FastAPI como framework backend
SQLAlchemy como ORM
PostgreSQL como base de datos
Pydantic para validación de datos
Uvicorn como servidor ASGI

Configuración de entorno

El archivo .env contiene la cadena de conexión a base de datos:

DATABASE_URL=postgresql://postgres:TU_PASSWORD@localhost:5432/aulanomina

config.py carga esta variable y la expone al resto del sistema.

Conexión a base de datos

db.py define:

- engine de SQLAlchemy
- SessionLocal para gestión de sesiones
- Base como clase base de los modelos

Esto permite trabajar con la base de datos de forma estructurada y reutilizable.

Modelos implementados

User
Modelo base preparado para futuras funcionalidades de autenticación y control de roles.

Campos principales:
id
email
password_hash
role
is_active
created_at

Employee
Primer modelo funcional del sistema, orientado a la gestión de trabajadores.

Campos:
id
employee_code (único)
first_name
last_name
dni (único)
email (único)
phone
birth_date
hire_date

status (active / inactive)
created_at
created_by (relación con users)

Esquemas de datos

Se definen tres esquemas principales:

- *EmployeeCreate*
  Define los datos necesarios para crear un empleado.
- *EmployeeUpdate*
  Define los datos utilizados en actualizaciones.
- *EmployeeResponse*
  Define la estructura de salida de la API, incluyendo identificador y fecha de creación.

Lógica de negocio

Se implementa una capa CRUD separada que gestiona el acceso a datos:

- *create_employee*
  Crea un nuevo empleado.
- *get_employees*
  Devuelve únicamente empleados activos.
- *get_employees_all*
  Devuelve todos los empleados sin filtrar.
- *get_employee*
  Obtiene un empleado por su id.
- update_employee
  Actualiza los datos de un empleado existente.
- soft_delete_employee
  Realiza una baja lógica cambiando el estado a "inactive".

Validaciones implementadas

Antes de crear o actualizar un empleado se valida:

- unicidad de employee_code
- unicidad de dni
- unicidad de email (si está informado)

En caso de duplicado se devuelve un error HTTP 400.

Endpoints disponibles

- GET /
  Verificación del estado del backend.
- POST /employees
  Creación de empleados.
- GET /employees
  Listado de empleados activos.
- GET /employees/all
  Listado completo de empleados.
- GET /employees/{employee_id}
  Obtención de un empleado concreto.
- PUT /employees/{employee_id}
  Actualización de datos de empleado.
- DELETE /employees/{employee_id}
  Baja lógica del empleado.

Ejemplo de creación

{
"employee_code": "EMP001",
"first_name": "Juan",
"last_name": "Pérez",
"dni": "12345678A",
"email": "juan@example.com"
}

Comportamiento de baja lógica

El endpoint DELETE no elimina registros físicamente.
Se actualiza el campo status a "inactive", permitiendo mantener trazabilidad y coherencia con sistemas reales de gestión laboral.

Acceso a la API

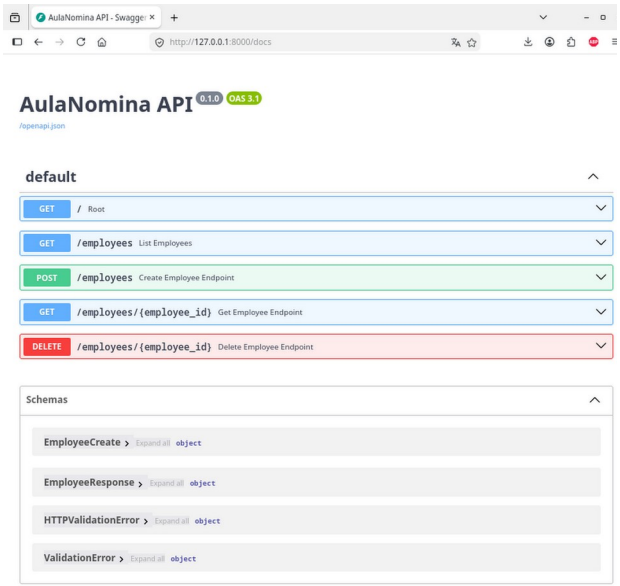
Servidor:
http://127.0.0.1:8000

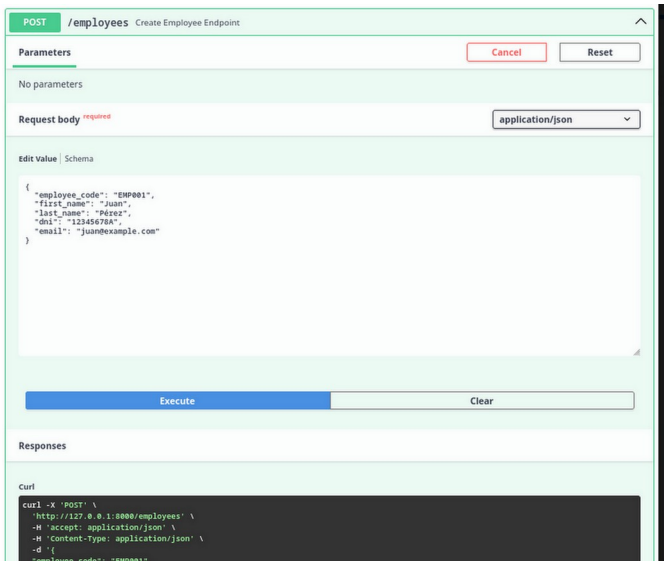
Documentación interactiva:
http://127.0.0.1:8000/docs

Capturas de pantalla

```
[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
(venv) alejandro@fedora:~/Documentos/AulaNomina/backend$ pip list
Package           Version
----------------- -------
alembic           1.18.4
annotated-doc     0.0.4
annotated-types   0.7.0
anyio             4.13.0
click             8.3.2
fastapi           0.135.3
greenlet          3.4.0
h11               0.16.0
idna              3.11
Mako              1.3.10
MarkupSafe        3.0.3
pip               25.1.1
psycopg2-binary   2.9.11
pydantic          2.12.5
pydantic_core     2.41.5
python-dotenv     1.2.2
SQLAlchemy        2.0.49
starlette         1.0.0
typing_extensions 4.15.0
typing-inspection 0.4.2
uvicorn           0.44.0
(venv) alejandro@fedora:~/Documentos/AulaNomina/backend$
```

```
typing_extensions 4.15.0
typing-inspection 0.4.2
uvicorn           0.44.0
(venv) alejandro@fedora:~/Documentos/AulaNomina/backend$ uvicorn app.main:app --reload
INFO:     Will watch for changes in these directories: ['/home/alejandro/Documentos/AulaNomina/backend']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [1127211] using StatReload
INFO:     Started server process [1127213]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```





Conclusión

Se ha construido una base backend sólida con conexión real a base de datos y un módulo completo de gestión de empleados.

El sistema permite:

- persistencia real de datos
- validación de negocio básica
- operaciones CRUD completas
- simulación realista de gestión de trabajadores

Este sprint establece una base firme para evolucionar hacia funcionalidades más complejas.

Siguiente paso

Sprint 2 — Contracts (gestión de contratos laborales)